

■■ Aavedan Setu — Internal System Workflows & Architectural Blueprint

This document details the internal technical workflows, database operations, algorithms, and system integrations of **Aavedan Setu**. It serves as an architectural blueprint for understanding how components communicate across the **React 19 Frontend**, **Django DRF Backend**, and **FastAPI AI Microservice**.

■■ System Interactions Overview

```
sequenceDiagram
    autonumber
    actor Citizen
    participant FE as React 19 Frontend (Port 5173)
    participant BE as Django DRF Backend (Port 8000)
    participant AI_MS as FastAPI AI Microservice (Port 8010)
    participant DB as PostgreSQL / SQLite Database
    participant SMTP as SMTP Mail Server

    %% Google OAuth 2.0 Login
    rect rgb(240, 248, 255)
        Note over Citizen, BE: 1. Google OAuth 2.0 Authentication Flow
        Citizen->>FE: Click "Continue with Google"
        FE->>FE: Open official Account Chooser Popup
        FE->>BE: POST /api/auth/google/ {token}
        BE->>BE: Verify token with google-auth library
        BE->>DB: get_or_create User profile
        BE->>FE: Return SimpleJWT tokens (Access & Refresh)
    end

    %% Conversational Grievance AI
    rect rgb(255, 250, 240)
        Note over Citizen, AI_MS: 2. Conversational Grievance NLP Orchestration
        Citizen->>FE: Submit chat/voice message ("pani asuni...")
        FE->>BE: POST /api/ai/chat/ {message, session_id}
        BE->>BE: Run Text Preprocessor & Intent Detector
        BE->>BE: Run Location Extractor & Fuzzy Matching
        BE->>AI_MS: Classify text/image (Groq Primary / Gemini Backup)
        AI_MS-->>BE: Return Category, Dept, Priority, Confidence
```

```
    BE->>BE: Decision Engine: Determine NextAction (e.g. ASK_ADDRESS)
    BE-->>FE: Return AI reply and structured session state
end

%% Email Preview & Dispatch
rect rgb(245, 255, 250)
    Note over Citizen, SMTP: 3. Professional Email Dispatch & DB Insertion
    Citizen->>FE: Approve Draft & click "Send Email"
    FE->>BE: POST /api/ai/chat/send-email/ {session_id}
    BE->>BE: Look up office contact email from location & dept
    BE->>BE: Draft professional letter via AI Microservice
    BE->>SMTP: Dispatch email with inline base64 evidence (CIDs)
    BE->>DB: Save grievance ticket (Status: pending)
    BE-->>FE: Return dispatch confirmation
end

%% Auto-Clustering & Priority Scoring
rect rgb(255, 240, 245)
    Note over BE, DB: 4. Complaint Stacking & Civic Budgeting Engine
    BE->>DB: Query pending tickets (group by Category + District)
    BE->>BE: Run Auto-Clustering (3-Tier Rule Matrix)
    BE->>DB: get_or_create CivicProject proposal
    BE->>BE: Recalculate 100-Point Weighted Priority Score
    BE->>BE: Recalculate relative District Priority Share (%)
    BE->>DB: Save updated scores and projects
end

%% Verification Ledger
rect rgb(245, 245, 245)
    Note over Citizen, BE: 5. Officer Proof & Citizen Verification Ledger
    actor Officer
    Officer->>FE: Toggle Switch to Officer Mode
    Officer->>FE: Upload "After Repair" geotagged photo
    FE->>BE: POST /api/complaints/projects/<id>/resolve/
    BE->>DB: Cascade status "IN_EXECUTION" & proof to associated complaints
    Citizen->>FE: Inspect Side-by-Side ledger (Before vs After)
    Citizen->>FE: Approve or Reject Proof
    FE->>BE: POST /api/complaints/projects/<id>/verify/
    BE->>DB: In case of 3 approvals -> status "COMPLETED" & complaints "RESOLVED"
    BE->>SMTP: In case of 3 rejections -> Re-open ticket & alert resolving officer
end
```

■ 1. Google OAuth 2.0 Authentication Workflow

The system implements a secure single-sign-on (SSO) workflow ensuring **frictionless onboarding** for citizens while validating their identity to prevent bot spam.

A. Frontend Verification (Login.jsx)

1. **Initiation:** The user clicks the "Continue with Google" button, triggering the custom hook `googleOAuthLogin` initialized via `@react-oauth/google`'s `useGoogleLogin`.
2. **Account Chooser:** An official Google account chooser popup opens, displaying all logged-in Google accounts.
3. **Information Retrieval:** Upon user selection, Google returns a temporary credential (`access_token` or `id_token`). The frontend queries Google's userinfo endpoint `https://www.googleapis.com/oauth2/v3/userinfo` using the token to fetch verified details:
 - Email address
 - Full name
4. **Backend Handoff:** The frontend sends the credentials to the backend via:

POST `/api/auth/google/` with `{ token: tokenResponse.access_token, email: googleUser.email, name: googleUser.name }`

5. **Demo Resiliency Fallback:** If the Google Client ID is unconfigured (returns `dummy`), the page detects this and handles authentication using a predefined sandbox profile (`demo.citizen@gmail.com`). This ensures that evaluation/testing is never blocked by API configuration issues.

B. Django Backend Token Authentication (views.py)

1. **Verification:** Inside `[GoogleAuthView](file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/accounts/views.py#L247-L303)`, the backend receives the Google token and verifies its authenticity using Google's official `id_token.verify_oauth2_token()` or fallbacks to the tokeninfo REST API (`https://oauth2.googleapis.com/tokeninfo`).
2. **Registration/Login:** The backend calls `User.objects.filter(email=email).first()`.
 - If the user exists: logs them in.
 - If not: registers a new account using the extracted verified email, full name, and generates a random unique phone number.
3. **SimpleJWT Generation:** The backend generates a pair of JSON Web Tokens (`access_token` and `refresh_token`) using SimpleJWT:

```
refresh = RefreshToken.for_user(user)
```

4. **Context Injection:** React receives the tokens, calls `loginWithTokens(access, refresh, user)` to update the global `AuthContext`, and redirects the browser to `/dashboard`.

2. Complaint Stacking & Civic Budgeting Engine

To prevent municipal budget wastage on duplicate work orders, Aavedan Setu features an **automatic clustering and democratic budgeting engine** that stacks related complaints into consolidated **Civic Projects**.

A. The 3-Tier Clustering Rule Matrix

Every time the civic budgeting dashboard is queried or a complaint status is modified, the backend runs `[recalculate_civic_projects()]`(file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/complaints/views.py#L405-L581). Individual tickets are grouped by their spatial-categorical coordinates: **(district_id, category_id, sub_issue_key)**.

The cluster is only promoted to a community-wide **CivicProject** if it satisfies the following matrix:

Scenario	Number of Tickets (\$R\$)	Upvotes/Support Count (\$L\$)	Promotion Status
---	---	---	---
High Ticket Volume	\$\ge 3\$ complaints	Any	Promoted
High Community Upvotes	\$2\$ complaints	\$\ge 2\$ upvotes	Promoted
Urgent Citizen Alert	\$1\$ complaint	\$\ge 3\$ upvotes	Promoted
Insufficient Concern	\$1\$ or \$2\$ complaints	\$< 2\$ upvotes	Left as individual ticket

B. Dynamic Cost Estimation Model

To prevent cost inflation (e.g., multiplying a street light repair cost of ₹25K by 100 duplicate tickets to produce a ₹25 Lakh budget), the system uses a **scalable cost-bracket formula**:

$$\text{Project Cost} = \text{Base Cost} + \min(R - 3, 10) \times ₹5,000$$

Where:

- **Base Cost:** Determined dynamically from the single source of truth `[KnowledgeRetriever]`(file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/ai/services/knowledge_retriever.py) (e.g., Road Collapse = ₹4.5L, Potholes = ₹1.25L, Transformer Repair = ₹85K, Drainage Overhaul = ₹60K).
- **Proportional Scope Scale:** Adds ₹5,000 per additional ticket up to 10 tickets, ensuring costs remain bound to realistic scope thresholds.

C. The 100-Point Weighted Priority Score Algorithm

Once a **CivicProject** is created, its priority order is computed dynamically using three criteria:

$$\text{Priority Score} = (0.40 \times S) + (0.35 \times D) + (0.25 \times V)$$

- 1. **Severity Score (\$\$\$)** [Weight: 40%]: Evaluated from the highest severity ticket in the cluster:
 - **CRITICAL** = 100 points
 - **HIGH** = 75 points
 - **MEDIUM** = 50 points
 - **LOW** = 25 points
- 2. **Demand Score (\$D\$)** [Weight: 35%]: Measures community density reporting. It is normalized relative to the most active project in the same district:

$$D = \frac{R + L}{\max(R + L)_{\text{district}}} \times 100$$

- 3. **Democratic Vote Score (\$V\$)** [Weight: 25%]: Normalized relative to the maximum direct funding votes received by any project in the district:

$$V = \frac{\text{Direct Project Votes}}{\max(\text{Votes})_{\text{district}}} \times 100$$

D. District Priority Share (%)

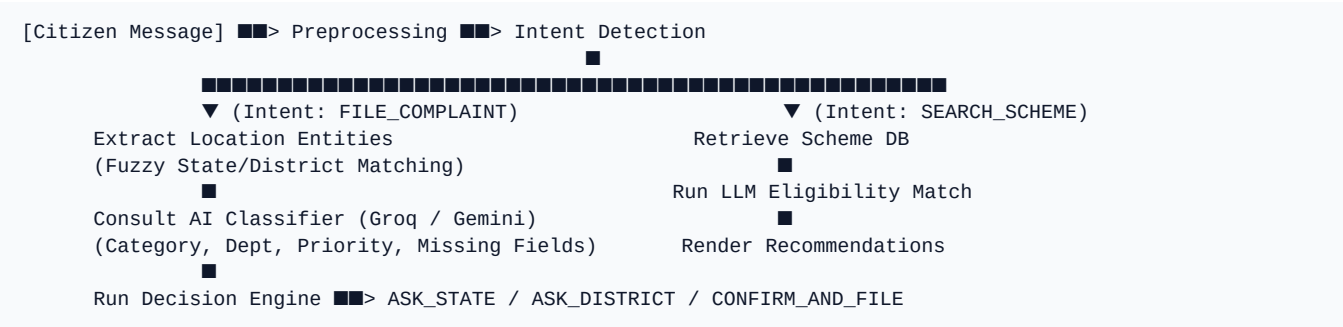
Every project is allocated a real-time relative percentage weight inside its district. This instructs municipal officers on how to partition the district’s unallocated capital (drawn from the **5 Crore DUDA Pool**):

$$\text{Priority Share (\%)} = \frac{\text{Project Score}}{\sum \text{All District Project Scores}} \times 100$$

This guarantees that high-safety hazards (e.g. fallen high-tension wires) instantly rank above highly-voted minor repairs, eliminating democratic bias.

3. AI Conversational Filing & NLP Orchestrator Flow

The conversational AI assistant (**Aavedan Saathi**) acts as an intelligent router guiding citizens to file structured complaints.



Step 1: Pre-processing & Intent Classification

- **[TextPreprocessor]**(file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/ai/services/text_preprocessor.py): Sanitizes input text, converts to lowercase, and maps Hindi/Hinglish verbs (e.g. **kut te**, **paani**, **naali**, **sadak**) to clean english keywords.
- **[IntentDetector]**(file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/ai/services/intent_detector.py): Resolves whether the citizen is greeting (**GREETING**), checking status (**TRACK_COMPLAINT**), looking up offices (**OFFICE_LOOKUP**), searching welfare schemes (**SEARCH_SCHEME**), or filing a new complaint (**FILE_COMPLAINT**).

Step 2: Location Extraction & Fuzzy Correction

- **[LocationExtractor]**(file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/ai/services/location_extractor.py): Parses inline location tags (e.g. "*water clogging in cutack*"). It strips civic words like "water", "street", "light", and feeds the remaining terms to a fuzzy string matcher (**difflib.get_close_matches** with 70%+ threshold) comparing against all 36 States and 700+ Districts:
 - **cutack** → **Cuttack (Odisha)**
 - **madhepur ii** → **Madhepura (Bihar)**
 - If GPS coordinates are active, Leaflet reverse-geocodes the coordinates and auto-fills State/District metadata.

Step 3: Classification & Department Resolution (The 3-Tier Cascade)

To guarantee 100% availability regardless of API quotas or network latency, the orchestrator invokes a cascading failover architecture:

1. **Tier 1 (Groq Cloud Qwen-27B)**: Sent to the ultra-fast Groq endpoint to parse text and extract structured JSON (category, department, priority, suggested items).
2. **Tier 2 (Google Gemini Flash Backup)**: If Groq hits rate limits (HTTP 429), the API router automatically catches the exception, switches keys using a rotating 5-key Gemini API pool, and retries request generation.
3. **Tier 3 (Offline KnowledgeRetriever Engine)**: If both external APIs are unreachable, the system executes an offline local regex rules engine mapping 74 civic sub-issue keys (e.g. "sewage leakage" → Drainage Board, High Priority, ■35K cost).

Step 4: Next-Action State Machine

The **[DecisionEngine]**(file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/ai/services/decision_engine.py) determines what information is missing to successfully register the ticket.

- If State is missing → returns **ASK_STATE**
 - If District is missing → returns **ASK_DISTRICT**
 - If Street Address is missing → returns **ASK_ADDRESS**
 - If all parameters are satisfied → returns **CONFIRM_AND_FILE**
-

■ 4. Official Email Dispatch System

Once a grievance is confirmed, the citizen can click **"Send Email"**. This triggers the backend to compile a formal petition to the corresponding authority.

A. Recipient Lookup

The `[OfficeFinder]`(`file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/ai/services/office_finder.py`) queries the database to locate the specific local division office handling the complaint's department inside the citizen's district.

- **Example:** Water Supply complaint in Madhepura → queries `DepartmentOffice` for `Department = "Water Supply Board"` and `District = "Madhepura"`, resolving the exact officer's email (e.g. `ee.madhepura@phe.bihar.gov.in`).

B. Anonymous Dispatch Routing

If the citizen checks **"File Anonymously"**:

- The **From** address is routed through a masked system email (`no-reply@aavedansetu.gov.in`).
- The citizen's personal name and Gmail address are replaced in the letter body with *"Anonymous Citizen"*.
- To ensure the citizen still gets official updates, the backend places their email in the **BCC** headers and strips their identity from the main header fields.
- Non-anonymous complaints append the citizen's email directly to the **CC** field and sets their address as the **Reply-To** target.

C. Inline CID Evidence Embedding

Unlike typical email attachments that get ignored, photos uploaded by the citizen are embedded inline in the HTML body.

1. The backend parses uploaded images and generates unique Content-IDs (CIDs) e.g. `<evidence_12>`.
2. The HTML body references the CID inline: ``.
3. The image is attached as a MIME image element. When the officer opens the mail, the photos render side-by-side inside the petition text block.

■ 5. Welfare Schemes Recommendation Engine

The system contains an AI-powered matching framework linking citizens to state and central schemes.

1. **AI Matching Model:** The system pulls the scheme metadata database and feeds it along with the user's description (income, age, profession) to the FastAPI AI service `/api/v1/schemes/recommend`.
2. **Offline Rule Fallback:** If the AI service fails, the system runs keyword mappings.
3. **Residency Enforcement Filter:** If the user's profile state is *Bihar*, and the matched scheme requires *Odisha* residency (e.g., Biju Pucca Ghar Yojana), the engine marks the scheme as **Not Eligible** and attaches the explanation: *"Your state is Bihar, but this scheme is only for Odisha residents."*

■ 6. Officer Resolution & Citizen Verification Ledger

To close the loop and prevent paper-only resolutions, Aavedan Setu demands evidence-grounded verification.

A. Officer Proof Submission

1. An officer navigates to a **CivicProject** on the dashboard, switches to **Officer Mode (Demo)**, and uploads an "After Repair" proof image accompanied by remarks.
2. Under the hood, this triggers `[GroupProjectResolveView](file:///e:/checking/Done_AavedanSetu/Aavedan-Setu/backend/complaints/views.py#L615-L676)`:
 - Sets the `CivicProject.status` to **IN_EXECUTION**.
 - **Cascading Auto-Resolution**: Automatically copies the proof photo and remarks to the `after_image` and `resolution_remarks` fields of **ALL associated citizen complaints** linked to this project cluster.
 - Updates the status of all these complaints to **review** (Under Review).

B. Citizen Verification Ledger

All completed works appear on the public community ledger `/civic-budgeting` for verification.

- **Approval Flow**: When citizens inspect the side-by-side "Before vs After" images:
- If **3 distinct citizens** approve the proof photo → `CivicProject.status` transitions to **COMPLETED**.
- All linked individual complaints are automatically marked **RESOLVED** and closed.
- **Rejection Flow**: If **3 distinct citizens** reject the proof (flagging it as a fake photo, wrong location, or poor work quality):
 - The project status resets to **PROPOSED** (Pending).
 - All linked complaints are re-opened (status reset to **pending**).
 - An automated alert email is sent directly to the resolving officer:

```
[URGENT ALERT] Resolution Proof Rejected: Madhepura Road Project
The proof uploaded from your account was rejected by 3 citizens.
The project has been re-opened for rework.
```